

数据存储问题快速答案

问题: 数据存储于哪里？普通数据和向量数据都可以使用sqlite，如何使用sqlite存储向量数据？

快速答案

1. 数据存储位置

用户本地存储：

```
~/Library/Application Support/FlowySandbox/storage/  (macOS)
├─ flowy-database.db      ← 普通业务数据
├─ vector.db             ← 向量嵌入数据
├─ flowy-database.db-shm  ← WAL日志
├─ flowy-database.db-wal  ← 检查点
└─ files/                 ← 上传的文件
```

大小：

- flowy-database.db: ~100MB (用户数据)
- vector.db: ~1GB (百万向量)

2. 普通数据存储 (flowy-sqlite)

用途: 用户、工作区、文档、订阅、支付、使用日志

表结构示例:

```
users (uid, email, name, ...)
documents (oid, workspace_id, title, content, ...)
user_subscription (id, user_id, plan_type, status, ...)
ai_usage_log (id, user_id, feature_type, tokens_used, cost_usd, ...)
```

特点:

- ✓ ACID事务支持
- ✓ 复杂查询 (JOIN, 聚合)
- ✓ 外键约束
- × 不支持向量相似度搜索

3. 向量数据存储 (flowy-sqlite-vec)

用途: 语义搜索、推荐系统、内容相似度

如何存储向量:

```

-- 使用sqlite-vec扩展创建虚拟表
CREATE VIRTUAL TABLE af_collab_embeddings
USING vec0(
    workspace_id TEXT,
    object_id TEXT,
    fragment_id TEXT,
    content TEXT,
    embedding float[768] ← 向量列(768维)
);

-- 插入向量数据
INSERT INTO af_collab_embeddings VALUES (
    'ws_123',
    'doc_456',
    'para_789',
    '这是文档内容',
    [0.1, 0.2, ..., 0.768] ← 768个float值
);

-- 向量相似度搜索
SELECT * FROM af_collab_embeddings
WHERE embedding MATCH '[0.05, 0.15, ..., 0.765]'
ORDER BY distance
LIMIT 10; ← 返回最相似的10个

```

特点:

- ✓ KNN向量搜索 (K-Nearest Neighbors)
- ✓ 自动HNSW索引化
- ✓ 低内存占用
- ✓ 支持百万级向量
- × 不支持事务 (虚拟表限制)

PROF

在Phase 2B中的应用

架构

```

支付系统 (Phase 2B)
↓
记录AI使用:
ai_usage_log {
    user_id: "user_123",
    feature_type: "semantic_search", ← 向量操作
    tokens_used: 150,
    cost_usd: 0.0015,
    created_at: NOW()
}
↓

```

存储向量数据：

```
af_collab_embeddings {  
  embedding: [768维向量], ← 用于后续搜索  
  content: "原始文本"  
}
```

↓

计费 and 配额管理：

检查Pro用户的月度搜索限额

累计成本：+\$0.0015

计费模型

向量操作成本 = tokens × \$0.01/1K

示例：

- 1次语义搜索 (150 tokens) = \$0.0015
- 1次嵌入生成 (1000 tokens) = \$0.01
- 批量索引 (10000 tokens) = \$0 (免费)
- 相似度推荐 (500 tokens) = \$0.0025 (5折)

计划限制

Free用户： 100次向量操作/月

Pro用户： 10,000次向量操作/月

Team用户： 100,000次向量操作/月

(操作 = 搜索、生成、推荐)

(批量索引不计入限额)

🔗 SQLite向量实现方式

方式1: sqlite-vec扩展 (PuerceNote采用)

```
// Rust代码  
use flowy_sqlite_vec::VectorSqliteDB;  
  
let vec_db = VectorSqliteDB::new(storage_path)?;  
  
// 插入向量  
vec_db.insert_embedding(  
  workspace_id: "ws_123",  
  object_id: "doc_456",  
  embedding: vec![0.1, 0.2, ..., 0.768], // 768维  
  content: "文档内容"
```

```

)?;

// 向量搜索
let results = vec_db.semantic_search(
    query_embedding: &[0.05, 0.15, ..., 0.765],
    top_k: 10 // 返回最相似的10个
)?;
// → [(doc_456, 0.95), (doc_789, 0.87), ...]

```

优势:

- 零额外部署 (无Pinecone/Weaviate)
- 本地优先 (数据完全本地)
- 自动索引 (HNSW算法)
- 成本极低 (无SaaS费)

方式2: JSON存储 (简单但慢)

```

-- 不推荐:用JSON存储向量
CREATE TABLE embeddings (
    id TEXT PRIMARY KEY,
    content TEXT,
    embedding TEXT -- JSON数组
);

INSERT INTO embeddings VALUES (
    'id_1',
    '文本内容',
    '[0.1, 0.2, ..., 0.768]' -- 作为JSON字符串
);

-- 搜索需要计算每一行的距离 (很慢)
SELECT * FROM embeddings
WHERE json_extract(embedding, '$') LIKE '%0.1%' × 低效

```

PROF

问题:

- 每次搜索都要计算所有行的距离
- 100万向量的搜索需要秒级延迟
- 无法使用索引优化

方式3: BLOB二进制存储

```

// 存储为二进制
let embedding_bytes = embedding
    .iter()
    .flat_map(|f| f.to_le_bytes())

```

```
.collect::<Vec<u8>>();

// 插入
INSERT INTO embeddings (id, embedding_blob)
VALUES ('id_1', x'...'); -- 二进制格式

// 搜索：需要手动实现距离计算（更复杂）
```

问题:

- 需要手动实现向量距离计算
- 无法用SQL查询，需要应用层处理
- 性能不如sqlite-vec扩展

方式对比

特性	sqlite-vec	JSON	BLOB	Pinecone
部署	零	零	零	有
延迟	<50ms	>1s	中等	<50ms
可扩展	1M向量	10K	100K	无限
成本	\$0	\$0	\$0	\$20-1000/月
学习曲线	低	低	中	中
推荐指数	☆☆☆☆☆	☆☆	☆☆☆	☆☆☆☆

PuerceNote选择: sqlite-vec (最适合本地优先)

🔗 后续步骤 (进行下一步)

现在 (完成Phase 2B基础)

- ✓ Task 2B.Token+Price (完成) - 定价优化
- ✓ Task 2B.SQL (完成) - 数据库架构
- 🔗 Task 2B.Webhook (进行中) - 支付事件
- 🔗 Task 2B.Tests (待开始) - 集成测试

明天 (Day 2 - 支付集成)

- 🔗 Task 2B.Web3.A: WalletConnect (3h)
- 🔗 Task 2B.Web3.B: MetaMask (2h)
- 🔗 Task 2B.PayPal: Google/Apple Pay (2h)
- 🔗 Task 2B.Paddle: Alipay/WeChat (1h)

后天 (Day 3 - 向量计费)

- 🔑 Task 2B.Vector: 向量计费模块集成
1. 扩展ai_usage_log表 (添加向量字段)

2. 集成vector_billing.rs到webhook

3. 实现向量操作配额检查

4. 添加向量成本计算

📁 新增代码 (已提交)

文档

- DATA_STORAGE_ARCHITECTURE.md (420行)
 - 三层存储架构详解
 - SQLite向量实现细节
 - 与计费系统集成
 - 性能和成本分析

代码

- flowy-subscription/src/billing/vector_billing.rs (280行)

```
pub struct VectorUsageRecord;  
pub struct VectorQuotaStatus;  
pub struct VectorBillingService;  
  
// 支持跟踪向量操作  
track_semantic_search()  
track_embedding_generation()  
estimate_cost()
```

测试

- 11个单元测试
 - 成本计算验证
 - 配额管理检查
 - 不同plan的limits
 - 成本估算

💡 关键点总结

概念

解答

概念	解答
存储位置	本地: ~/Library/Application Support/FlowySandbox/storage/
普通数据	flowy-sqlite (flowy-database.db) - 关系型表
向量数据	flowy-sqlite-vec (vector.db) - vec0虚拟表
向量维度	float[768] (默认, 可配)
搜索方式	KNN相似度搜索 (自动HNSW索引)
大小极限	100万向量 (3GB磁盘)
搜索延迟	<50ms (1M向量)
与计费关系	ai_usage_log记录成本: \$0.01/1K
配额管理	Free 100, Pro 10K, Team 100K ops/月

推荐阅读

1. 详细架构: [DATA_STORAGE_ARCHITECTURE.md](#) (这个仓库中)
2. 向量计费: [flowy-subscription/src/billing/vector_billing.rs](#) (代码)
3. sqlite-vec文档: <https://github.com/asg017/sqlite-vec>
4. PuerceNote存储: [frontend/rust-lib/flowy-sqlite*/](#)

? 常见问题

Q: 向量数据会自动备份吗?

A: 是的,vector.db与flowy-database.db遵循相同的备份策略 (WAL文件)

Q: 可以分享向量搜索结果吗?

A: 结果内容可以共享,但向量本身是私有的(不会传送)

Q: 多个工作区的向量会混淆吗?

A: 不会,workspace_id作为分片键保证隔离

Q: SQLite向量扩展支持什么距离函数?

A: 默认欧几里得距离,支持余弦相似度

Q: 可以离线搜索吗?

A: 是的,完全本地搜索,无需网络

准备好进行下一步了吗?

现在完成:

-  支付系统基础 (Task 2B.Token+Price + SQL)
-  向量计费模块 (vector_billing.rs)
-  数据存储文档

下一步选择:

1. **继续Phase 2B Day 1:** 完成Webhook事件处理
2. **跳到Day 2:** 开始Web3支付集成
3. **深入向量搜索:** 实现完整的语义搜索功能